

REAL-TIME ANALYTICS, BI & PRESCRIPTIVE ANALYTICS

Exercise 8 - Grafana Live Monitoring and Alert Engineering

Grafana OSS + your Exercise 8 database

BEFORE YOU START

Exercise 8 must be finished and the feed simulator must be running in a second terminal. A Grafana panel over a static table teaches you nothing about refresh.

Download Grafana OSS from grafana.com/oss/grafana - the free, self-hosted edition. Default address after starting it is <http://localhost:3000>, default login admin / admin.

Every count in this handout was computed from the shipped feed. Your Grafana counts will match if your rule matches.

What you are building and why

You are going to build a rule that is correct and unusable, measure exactly how unusable it is, and then apply three mechanisms until it becomes a rule a NOC would actually keep switched on. The measurement is the exercise. A tuned rule that you cannot show a before-and-after count for is a guess.

Deliverables

- A Grafana dashboard on live data with auto-refresh, showing latency, throughput and downtime by cell.
- One alert rule with a threshold, a pending period and a mute timing.
- A before-and-after alert count, and the name of every incident that survives.

Step 1 - Connect Grafana to your database

SQLite needs a plugin; MySQL is built in. Pick the path that matches what you used in Exercise 9

If you used MySQL

1. In Grafana, go to Connections, then Data sources, then Add new data source, then MySQL.
2. Host: localhost:3306. Database: the schema holding kpi_readings. User and password: your MySQL credentials.
3. Click Save & test. You must see a green confirmation before continuing.

If you used SQLite

4. Go to Administration, then Plugins, search for the SQLite data source by Fr-Ser, and install it.
5. Add the data source and set Path to the absolute path of telecom_rt.db - for example C:\\telecom\\telecom_rt.db.
6. Save & test.

IF THE PLUGIN WILL NOT INSTALL

You are probably offline or behind a proxy. Ask the trainer for the offline plugin bundle, or move your table into MySQL - the SQL in this handout runs on both with only the date functions changed.

Step 2 - Build the panel set a NOC would watch

Four panels. Each one answers a question you can say out loud. Set the dashboard time range to Last 6 hours and the refresh interval to 10s in the top-right control.

Panel	Question it answers	Type	Query shape
Latency by cell	Is any cell slowing down right now?	Time series	AVG(latency_ms) grouped by 15-min window and cell_id

Panel	Question it answers	Type	Query shape
Throughput by cell	Is any cell losing capacity?	Time series	AVG(throughput_mbps), same grouping
Cells in downtime	Is anything off air?	Stat	COUNT(DISTINCT cell_id) WHERE downtime_min > 0
Worst cells now	Which cells do I look at first?	Table	Latest interval, ordered by latency_ms descending

Latency panel query (SQLite). Grafana needs a column literally named time for the x-axis.

```
SELECT
  strftime('%s',
    substr(reading_ts,1,14) ||
    printf('%02d', (CAST(substr(reading_ts,15,2) AS INTEGER)/15)*15) || ':00'
  ) AS time,
  cell_id AS metric,
  AVG(latency_ms) AS value
FROM kpi_readings
WHERE latency_ms IS NOT NULL
GROUP BY time, cell_id
ORDER BY time;
```

The same query on MySQL

```
SELECT
  UNIX_TIMESTAMP(
    FROM_UNIXTIME(FLOOR(UNIX_TIMESTAMP(reading_ts)/900)*900)
  ) AS time,
  cell_id AS metric,
  AVG(latency_ms) AS value
FROM kpi_readings
WHERE latency_ms IS NOT NULL
GROUP BY time, cell_id
ORDER BY time;
```

NOTE WHAT IS MISSING FROM THAT QUERY

There is no MAX(latency_ms). For a wallboard the average is the right choice, because a wallboard is read continuously and a peak-driven line is unreadable.

For the alert rule in Step 4 you will need the peak, not the average. Same data, different question, different aggregation. Do not reuse the panel query as the alert query without thinking about it.

Step 3 - Create the naive rule and count it

Before Grafana, count it in SQL. You need a number you can defend, and the Grafana alert history is harder to count than a query.

```
SELECT COUNT(*) AS breaching_readings
FROM (SELECT DISTINCT cell_id, reading_ts, latency_ms, downtime_min
      FROM kpi_readings)
WHERE latency_ms > 60 OR downtime_min > 0;
```

Expected output (on the seeded rows, before your simulator adds any)

```
breaching_readings : 88
```

TWO NUMBERS, AND WHY THEY DIFFER

88 is the count on the de-duplicated table you built in MoP 9.1. On the raw file, before de-duplication, the same rule breaches 103 times - 15 of the 26 duplicate readings were themselves breaching. De-duplication is therefore the first stage of alert reduction, not merely a data-cleaning nicety. A double-writing collector inflates your alert volume by 17% before you have tuned anything.

Now build it in Grafana.

7. Go to Alerting, then Alert rules, then New alert rule. Name it: Cell degraded - naive.
8. Query A: the latency query from Step 2, but with MAX(latency_ms) instead of AVG, and no GROUP BY cell_id in the reduce step yet.
9. Set the threshold expression: IS ABOVE 60.
10. Set Pending period to 0s. Evaluate every 1m.
11. Save it, let it run, and watch the alert list. Record how many instances go into Firing state.

Step 4 - Add suppression: the maintenance windows

READ THIS BEFORE YOU CONFIGURE ANYTHING

Three of the cells in this feed were switched off on purpose by an engineer holding a change ticket. Those windows are listed in maintenance_windows.csv. Alerting on them is not a false positive in the statistical sense - the cell really was down. It is worse than that: you have paged an engineer to tell them about something they are currently doing.

Cell	Window start	Window end	Reason
CELL-MUM-02	2026-08-04 02:00	2026-08-04 04:00	Software upgrade - BTS baseband
CELL-KOL-03	2026-08-05 01:30	2026-08-05 03:00	Antenna tilt optimisation
CELL-BLR-02	2026-08-06 02:00	2026-08-06 03:30	Transport link migration

In Grafana: Alerting, then Notification policies, then Mute timings, then Add mute timing. Create one covering 01:00–04:30, and restrict it with a label matcher so it applies only to the three cells above - not to every cell in that window.

Verify the effect in SQL first

```
SELECT COUNT(*) AS after_suppression
FROM (SELECT DISTINCT cell_id, reading_ts, latency_ms, downtime_min
      FROM kpi_readings) k
WHERE (k.latency_ms > 60 OR k.downtime_min > 0)
AND NOT EXISTS (
  SELECT 1 FROM maintenance_windows m
  WHERE m.cell_id = k.cell_id
        AND k.reading_ts >= m.window_start
        AND k.reading_ts < m.window_end);
```

Expected output

after_suppression : 68 (20 readings suppressed)

LOAD THE WINDOWS INTO THE DATABASE

The query above assumes a maintenance_windows table. Load the CSV the same way you loaded the readings in MoP 9.1 - pandas to_sql is fine here, there is nothing to de-duplicate. In production this table is not a CSV. It is a live feed from the change management system, and the suppression rule reads it continuously. That integration is the single highest-value piece of alerting engineering most operators never build.

Step 5 - Add persistence

A breach that lasts one interval has self-healed before anyone could drive to site. Require the condition to hold for three consecutive intervals - 45 minutes at this grain - before anything is delivered.

In Grafana: set Pending period on the rule to 45m. The rule enters Pending on the first breach and only becomes Firing if the condition is still true 45 minutes later.

WHAT THIS DISCARDS

44 separate breach runs in this feed are shorter than three intervals. Every one of them would have paged somebody under the naive rule, and every one of them was over before a response was possible.

That is the difference between a rule that is correct and a rule that is useful.

Step 6 - Add hysteresis

A KPI sitting near the threshold will cross it repeatedly, producing a fire-clear-fire-clear sequence that generates a dozen notifications for one condition. Hysteresis separates the fire level from the clear level.

Setting	Value	Effect
Fire above	60 ms	The condition becomes true
Clear below	50 ms	The condition becomes false - a 10 ms dead band
Pending period	45 min	Three consecutive intervals before delivery
Keep firing for	15 min	Grafana's resolve delay, a second dead band in time

In Grafana: use two threshold expressions combined with a Math expression, or use the Recovery threshold field on the threshold expression if your Grafana version offers it. Both give the same behaviour.

Step 7 - Count the result and name every survivor

This is the deliverable. Record both numbers and the cell names.

Stage	Alerts	What was removed
Raw rule on the raw file	103	-
After de-duplication	88	15 duplicate breaching readings
After maintenance suppression	68	20 planned-outage readings
After 3-interval persistence	22	44 short runs that self-healed
After hysteresis grouping	3	Repeat notifications for the same condition

Your three surviving incidents

Cell	Started	What it was
CELL-BLR-03	2026-08-04 09:15	Unplanned outage - downtime recorded, no change ticket
CELL-DEL-03	2026-08-05 17:30	Progressive congestion, peaking at 143.77 ms
CELL-MUM-03	2026-08-06 19:15	Busy-hour congestion, self-resolving

THE ARITHMETIC TO QUOTE

180 alerts per week became 5 per week. That is a 34-fold reduction, and not one real incident was lost.

If you finish with zero alerts, your mute timing is too wide - check whether CELL-DEL-03 is still firing. Suppressing a real incident along with the planned ones is the failure this exercise is designed to make you feel.

Step 8 - Give each rule an owner

An alert with no owner is a log line. Fill this in for your rule before you call the exercise finished.

Field	Your answer
Severity (P1–P4)	
Who receives it	
Expected response time	
What they should do first	
What happens if nobody responds	
When this rule should be reviewed	

Completion checklist

#	Check	Evidence
1	Data source connected	Green Save & test, panels drawing
2	Dashboard refreshes live	Row count and lines advancing with the simulator
3	Naive count recorded	88 on your table, 103 on the raw file
4	Suppression scoped to three cells	68 after suppression, and no real incident lost
5	Persistence applied	Pending period 45m, 22 firing intervals
6	Three incidents, named	BLR-03, DEL-03, MUM-03
7	Rule has an owner	Step 8 table filled in